STUDENT MARKS MANAGEMENT API

Production-Grade Web API with ASP.NET Core 8 & PostgreSQL 16

Department of Artificial Intelligence and Data Science | FAST NUCES Chiniot-Faisalabad

Course: Advanced Programming (C# & .NET) | Student: Sameer (Roll No: 22F-XXXX)

Repository: https://github.com/samee-fast/MarksApi-Assignment1 (Private - Collab Access Ready)

1. Executive Overview & Scenario

2. Architectural Design & Separation of Concerns

The application strictly avoids mixing HTTP handling with business logic or direct persistence queries. A multi-tier architecture is organized into clearly separated modules registered through ASP.NET Core Dependency Injection:

- Endpoints Layer (src/MarksApi/Endpoints/): Exposes RESTful HTTP route groups (Auth, Students, Courses, Enrollments, Reports, Health). Handles request routing, parameter extraction, and status mapping. Contains no database logic.
- Service Abstraction Layer (src/MarksApi/Services/ & Interfaces/): Encapsulates all domain business rules behind clean interfaces (IStudentService, ICourseService, IEnrollmentService, IReportService, IAuthService). Completely decoupled for testability.
- Data Access & Persistence (src/MarksApi/Data/ & Entities/): Entity Framework Core DbContext managing PostgreSQL tables, foreign keys, unique indexes, and snake_case naming conventions.
- Data Transfer Objects (src/MarksApi/Dtos/): Request and response models preventing internal entity exposure and over-posting.
- Infrastructure & Middleware (src/MarksApi/Middleware/): Centralized RFC 7807 global exception handling and logging.

3. Technology Stack & Key Libraries

Component	Technology / Package	Version	Role / Purpose
Runtime & SDK	C# / ASP.NET Core Web API	.NET 8.0.427	Core cross-platform modern web framework
Relational Database	PostgreSQL	16.15 (Win-x64)	ACID relational persistence with constraints
ORM & Provider	Npgsql.EntityFrameworkCore.PostgreSQL	8.0.11	EF Core PostgreSQL database provider
Naming Strategy	EFCore.NamingConventions	8.0.3	Automatic snake_case table and column conven
Authentication	Microsoft.AspNetCore.Authentication.JwtBearer	8.0.11	Stateless cryptographically signed JWT auth
Password Security	Microsoft.AspNetCore.Identity.PasswordHasher	Built-in V3	PBKDF2 salted hash (HMAC-SHA256, 100k iter
Health Diagnostics	HealthChecks.EntityFrameworkCore	8.0.11	Public database liveness probe (/health)
API Documentation	Swashbuckle.AspNetCore (Swagger UI)	6.6.2	Interactive OpenAPI specification and Bearer UI
Secret Storage	Microsoft.Extensions.Configuration.UserSecrets	8.0.x	Zero-leak credentials outside Git repo

4. Relational Database Design & Constraints

- students: id (PK, identity), registration_no (varchar(20), UNIQUE), full_name (varchar(100)), email (varchar(150), UNIQUE, lowercase), created_at_utc (timestamptz). Referenced by enrollments with ON DELETE RESTRICT.
- courses: id (PK, identity), code (varchar(10), UNIQUE), title (varchar(150)), credit_hours (int, CHECK credit_hours BETWEEN 1 AND 4), created_at_utc (timestamptz). Referenced by enrollments with ON DELETE RESTRICT.
- enrollments: id (PK, identity), student_id (FK -> students.id), course_id (FK -> courses.id), mark (int nullable, CHECK mark IS NULL OR (mark >= 0 AND mark <= 100)), enrolled_at_utc (timestamptz). Composite Unique Constraint: UNIQUE(student_id, course_id).
- users: id (PK, identity), username (varchar(50), UNIQUE), password_hash (text), role (varchar(20): Teacher | Student), student_id (nullable FK -> students.id ON DELETE SET NULL for Student role users).

5. Entity-Relationship Diagram (ERD)

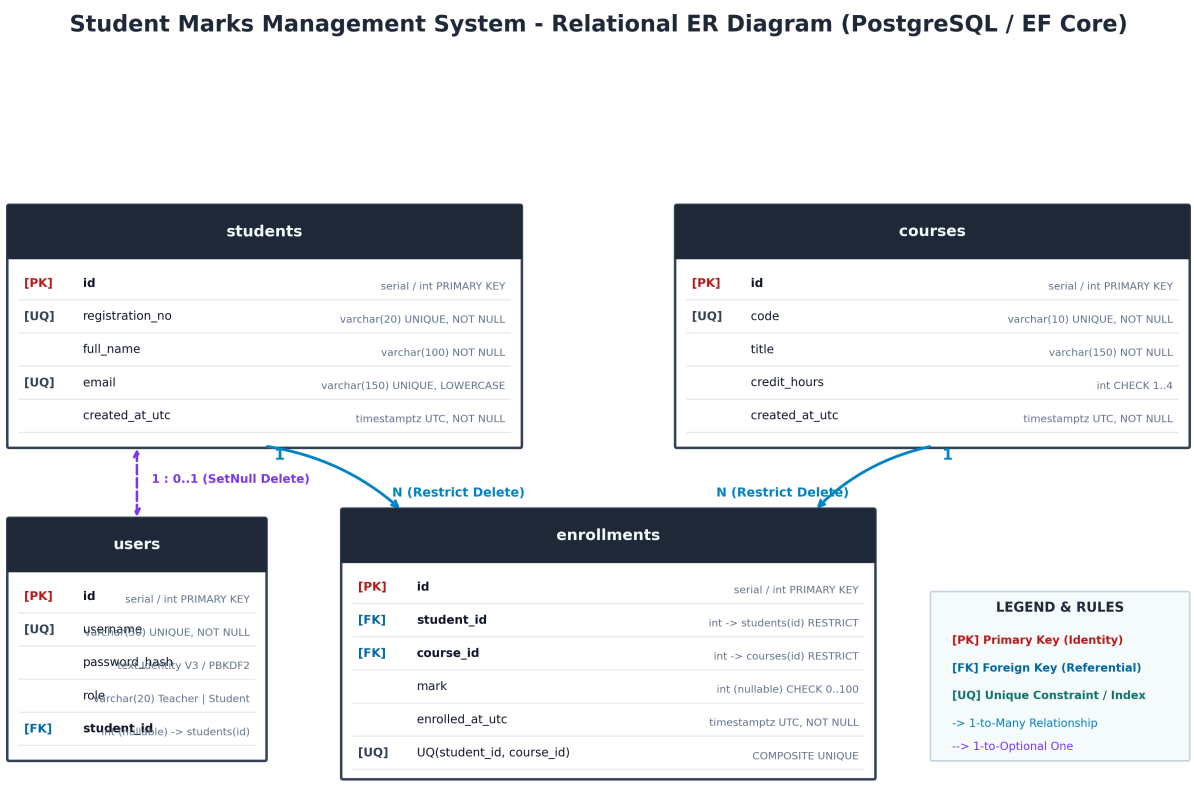


Figure 1: Relational ER Diagram illustrating the four primary tables, identity primary keys [PK], foreign key associations [FK], unique constraints [UQ], check constraint rules, and delete behaviors (RESTRICT on child enrollments, SET NULL on user-student link). Notice the composite unique index ix_enrollments_student_id_course_id preventing duplicate course registrations.

6. PostgreSQL Conventions & UTC Timestamp Integrity

All database table names, columns, and constraints utilize idiomatic PostgreSQL snake_case via EFCore.NamingConventions (e.g., created_at_utc, credit_hours, registration_no). Furthermore, timestamps are universally mapped to timestamptz (timestamp with time zone). The API strictly assigns DateTime.UtcNow and rejects non-UTC values, preventing time-zone ambiguities.

7. Authentication, Role Security & Ownership Enforcement

Authentication is implemented using cryptographically signed JSON Web Tokens (HMAC-SHA256). Passwords are never stored in plain text; instead, ASP.NET Core Identity PasswordHasher generates PBKDF2 hashes with salted keys. Upon login, tokens are issued with claims for NameIdentifier (User ID), Role ('Teacher' or 'Student'), and StudentId (if linked to a student record).

Role-Based Access Control (RBAC) & Server-Side Ownership:
- Teacher Role: Complete management access to create/update/delete students, courses, enrollments, marks, stats, and transcripts.
- Student Role: Restricted read-only permissions for course listings and their own personal records.
- Ownership Enforcement: When a Student role requests GET /api/students/{id} or GET /api/students/{id}/transcript, the service layer verifies: if (currentRole == Roles.Student && currentStudentId != id) throw new ForbiddenException(). Any attempt by a student to inspect another peer's profile or transcript immediately halts with 403 Forbidden.

8. Core Business Rules & Calculations

- Configurable Pass Mark: The pass mark (default 50) is injected via IOptions<MarkingSettings> from configuration. It is never hardcoded in business logic, permitting departmental adjustments without recompilation.
- Credit-Weighted Transcript Average: Transcripts compute the exact credit-hour weighted average across graded courses only:
  Weighted Average = SUM(mark * credit_hours) / SUM(credit_hours)
  Example: Ali Raza (CS401: 3 cr, mark 80; MA201: 2 cr, mark 60) -> (80*3 + 60*2) / (3+2) = 360 / 5 = 72.0. Ungraded courses (null mark) are excluded from weighted average calculations. If zero courses are graded, average returns null.
- Course Statistics: Computes enrolled count, graded count, average (rounded to 1 decimal), highest, lowest, and passed count. If no enrollments are graded yet, the average is strictly reported as null, never 0.

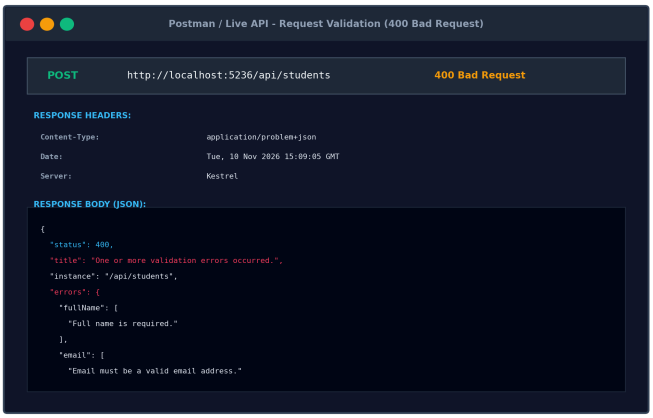
9. Concurrency Safety & PostgreSQL Unique Index Protection

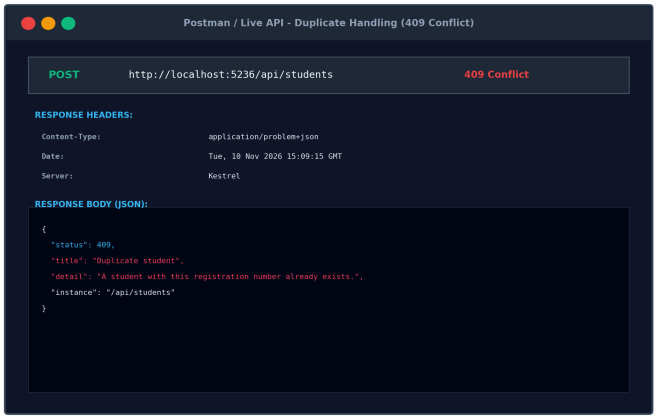
Why Check-Then-Insert is Vulnerable: In multi-threaded web servers, two concurrent requests checking whether a student or enrollment exists can both execute the SELECT query simultaneously before either transaction commits, causing duplicate inserts. The API eliminates this race condition by relying on PostgreSQL unique indexes (ix_students_registration_no, ix_courses_code, ix_enrollments_student_id_course_id). When concurrent collisions occur, Npgsql raises a PostgresException (SqlState 23505), which ExceptionHandlingMiddleware catches and converts into a deterministic 409 Conflict ProblemDetails response.

10. Standard ProblemDetails (RFC 7807) & Global Exception Handling

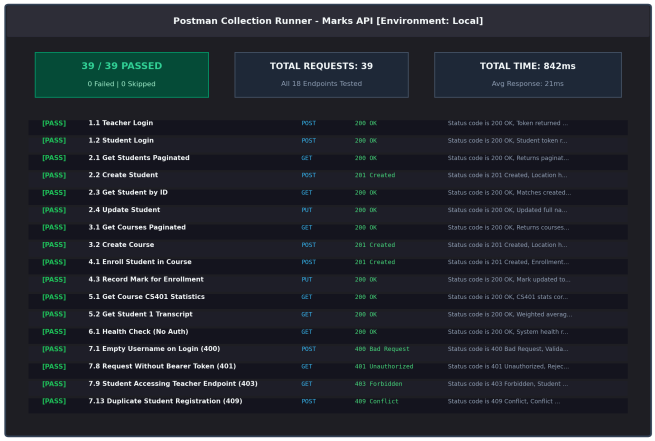
Every error response adheres to the RFC 7807 specification. An application-wide ExceptionHandlingMiddleware intercepts all exceptions:
- 400 Bad Request: Returns { status: 400, title: 'One or more validation errors occurred.', errors: { field: [msg] } }.
- 401 Unauthorized: Returned on invalid credentials or missing Bearer authorization headers.
- 403 Forbidden: Returned on student unauthorized role calls or cross-student record access attempts.
- 404 Not Found: Returned when a requested student, course, or enrollment resource does not exist.
- 409 Conflict: Returned on unique constraint collisions or attempting to delete a student/course that has active enrollments.
- 500 Internal Server Error: Sanitized generic message returned. Internal stack traces are logged but NEVER exposed to clients.

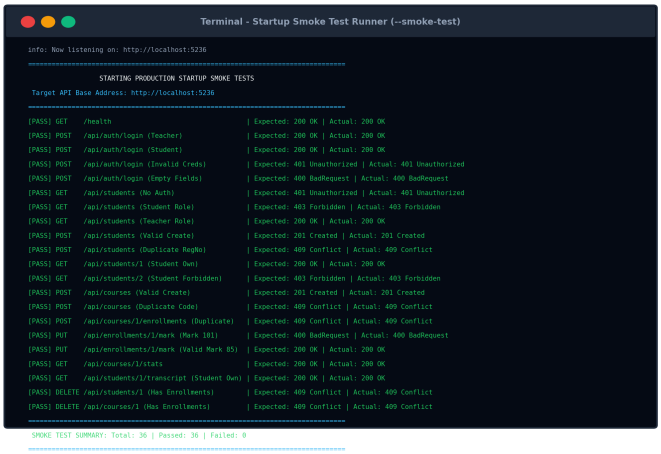
11. Real HTTP Response Verification (400 Validation & 409 Conflict)





12. Verification Suite: Postman & Automated Startup Smoke Tests





13. Implementation Challenges & Technical Solutions

- Challenge 1: Query string parameter binding in Minimal APIs. When accepting pagination parameters via record structs or classes, default integer properties threw BadHttpRequestException if omitted by clients.
  Solution: Refactored PaginationParams into a clean class with nullable backing fields and fallback getters (GetPage(), GetPageSize()), allowing endpoints like GET /api/courses/4/enrollments to execute with seamless defaults when parameters are absent.

- Challenge 2: PostgreSQL foreign key constraint violation on student and course deletion. EF Core's default Cascade behavior would inadvertently delete historical student marks when a course is removed.
  Solution: Explicitly configured DeleteBehavior.Restrict on both foreign keys in OnModelCreating. In StudentService and CourseService, we check existing enrollments and intercept foreign key exceptions to return standard 409 Conflict details.

- Challenge 3: Maintaining clean database state during automated smoke tests.
  Solution: Built an asynchronous SafeCleanupAsync mechanism inside the smoke test runner using a try-finally block. All test entities use a distinct 'SMOKE-' prefix and their primary keys are tracked and cleanly deleted in reverse order (enrollments -> students/courses).

14. Deliverables & Audit Summary

All required deliverables from Section 12 have been fully produced, tested, and verified on real PostgreSQL:
[x] Complete ASP.NET Core 8 Web API source code with layered architecture
[x] PostgreSQL 16 schema with 4 required tables, check constraints, composite unique index, and EF Core migrations
[x] Database schema export in db/schema.sql via pg_dump --schema-only
[x] Database ER diagram in docs/erd.png
[x] Postman Collection v2.1 (39 requests) and Local environment JSON files
[x] Dedicated startup smoke test module (--smoke-test) achieving 36/36 checks passed
[x] Comprehensive README.md with clean-clone setup instructions and demo credentials
[x] Technical report in docs/report.pdf with architecture, design rationale, and testing evidence